WEBXKEY

KV Saloon Booking & Management System

Phase 1 — Online Booking
Project & Technical Documentation

Client: KV Saloon

Prepared by: Webxkey

Version 0.1 (draft) · 29 October 2026

Contents

Overview	Project overview: all phases	3
Section 1	Product Requirements Document (PRD): Phase 1 Online Booking	5
Section 2	Business Rules: Booking & Availability	9
Section 3	Architecture & Project Structure	13
Section 4	Database Schema (Phase 1)	16
Section 5	API Specification (Phase 1)	21
Section 6	User Flows & Screens	24
Section 7	Roadmap & Task Backlog (Phase 1)	27
Section 8	Coding Standards, Git Workflow & Review	31
Section 9	Test Plan & UAT Checklist (Phase 1)	33
Section 10	Deployment & Operations	35
Section 11	Open Questions for KV Saloon	37
Section 12	AI Agent Playbook (how Webxkey builds with agents)	38
Appendix A	AGENTS.md — KV Saloon Booking System	41
Appendix B	ADR-001: Laravel modular monolith with Inertia + React	43
Appendix B	ADR-002: Prevent double booking with ordered row locks	44

OVERVIEW

Project overview: all phases

KV Saloon's system is delivered in phases on one platform. **Phase 1 (Online Booking) starts first** and is documented in full in this pack. Later phases build on the same codebase and database.

Phase	Name	Scope	Duration	Status
1	Online Booking	Services and pricing, staff and schedules, online booking with automatic staff assignment, front desk calendar, staff panel, SMS / email confirmations and reminders	5–7 weeks	**Starting now**
2	Customers	Customer profiles, visit history and notes, booking-based daily sales report	2–3 weeks	Planned
3	Staff & Payroll	Attendance, leave, commission rules, salary calculation, payslips	3–4 weeks	Planned
4	Inventory, Expenses & Reports	Stock and suppliers, expenses, profit & loss, staff performance, owner dashboard	3–4 weeks	Planned
5	Growth (optional)	Loyalty and packages, online deposits, WhatsApp marketing, multiple branches, mobile app	To be scoped	Optional

Phase 1 at a glance

Milestone	Outcome	Week
M0 Foundation	Project set up, CI, roles, shared building blocks	1
M1 Data & catalog	Database, demo data, services and staff admin	2
M2 Availability engine	Correct time slots for any service and staff	3
M3 Booking core	Safe booking (no double booking), reschedule, cancel, statuses	4
M4 Customer portal	Full online booking on a phone	5
M5 Front desk & staff	Calendar, walk-ins, drag-and-drop, staff panel	5–6
M6 Notifications	SMS / email confirmations and reminders	6
M7 Launch	Client testing (UAT), go-live, training	7

How to read this document

Section	For
1. Product requirements	Client and team: what Phase 1 delivers, sign-off
2. Business rules	Exact booking behaviour (highest priority)
3–6. Architecture, database, API, flows and screens	Developers and AI agents
7. Roadmap & tasks	Day-to-day build plan
8–10. Standards, testing, deployment	Quality and go-live
11. Open questions	Answers needed from KV Saloon

Section	For
12. AI agent playbook	How Webxkey builds with AI agents
Appendix	Agent rules (AGENTS.md) and decision records

SECTION 1

Product Requirements Document (PRD): Phase 1 Online Booking

Project	KV Saloon Booking & Management System
Client	KV Saloon
Vendor	Webxkey
Phase	1 — Online Booking
Version	0.1 (draft)
Status	Awaiting client sign-off

1. Goal

Let KV Saloon customers book appointments online 24/7, with each booking automatically assigned to an available, qualified staff member, and give the salon a calendar to manage all bookings and walk-ins.

Success measures (first 3 months after go-live)

- At least 40% of appointments are booked online
- No double bookings
- No-show rate reduced through reminders (baseline to be measured in month 1)
- Front desk can create a walk-in booking in under 30 seconds

2. Users & roles

Role	Description	Access
Customer	Public visitor	Booking portal; manage own booking via secure link
Owner	Salon owner	Everything, including settings and staff
Receptionist	Front desk	Calendar, bookings, customers; no settings
Staff	Stylist / beautician	Own schedule and own appointments only
Super admin	Webxkey	Support access, salon setup

3. In scope (Phase 1)

Customer booking portal (mobile-first, PWA)

- Browse services by category with price and duration
- Select one or more services
- Choose a specific staff member or "Any staff"
- See available dates and time slots
- Enter name, mobile number, optional email and note
- Verify mobile by SMS OTP (configurable on/off)
- Booking confirmation screen + SMS/email with reference number

- Manage booking via secure link: view, reschedule, cancel (within policy)

Admin / front desk dashboard

- Day/week calendar with one column per staff member
- Create booking (phone/walk-in) with same availability rules; staff override allowed for owner/ receptionist
- Drag and drop to move or reassign (re-validated server-side)
- Change status: confirmed → checked-in → completed; cancelled; no-show
- Search bookings by name, phone, reference
- Manage services & categories (name, duration, buffer, price, active, online-bookable)
- Manage staff (profile, photo, skills, weekly schedule, breaks, time off, display order, bookable online)
- Salon settings: opening hours, holidays, slot interval, lead time, booking window, cancellation window, assignment strategy, OTP on/off, reminder times
- Users & roles (owner invites receptionist/staff)

Staff panel (mobile-friendly)

- Today / upcoming appointments
- Customer name, services, notes
- Mark checked-in / completed (if allowed by settings)

Notifications

- Booking confirmation (SMS + email)
- Reminder 24h and 2h before (configurable)
- Reschedule and cancellation notices
- New booking alert to staff member (SMS or in-app)

4. Out of scope (later phases)

POS/billing, payments and deposits, payroll and commissions, inventory, expenses, profit reports, loyalty, multi-branch UI, native mobile app, WhatsApp (planned Phase 1.1 if client confirms).

5. User stories & acceptance criteria

Customer

US-01 Browse services As a customer I want to see services with prices and durations so I can choose what I need.

- Only active, online-bookable services are shown, grouped by category
- Each shows name, duration, price (LKR), optional description

US-02 Choose staff or "Any staff"

- Only staff who are active, bookable online, and skilled for **all** selected services are listed
- "Any staff" is always the first option

US-03 See available slots

- Dates from today + lead time up to booking window (default 30 days)
- Days with no availability are disabled
- Slots follow business rules §3–§5 and refresh if older than 2 minutes

- With "Any staff", a slot is shown if at least one eligible staff is free

US-04 Confirm booking

- Required: name, mobile (Sri Lanka format, normalised to +94), consent checkbox
- If OTP enabled, booking is created only after OTP verified
- If the slot was taken meanwhile, show "This time was just booked" and refresh slots (HTTP 409)
- On success: reference (e.g. `KV-7H3K9Q`), date, time, staff name, services, total price, manage link

US-05 Manage booking

- Secure signed link, no login needed
- Reschedule / cancel allowed only before the cancellation window (default 3 hours before start)
- After window, show salon phone number

Front desk

US-10 Calendar — day view default, columns per staff; booked items coloured by status; staff time off and breaks shaded.

US-11 Create booking — choose customer (search by phone or create new), services, staff or any, time. Receptionist may force-book outside availability with a warning; the override is logged.

US-12 Move/reassign — drag to new time/staff; server re-validates; customer notified if time changed.

US-13 Status changes — check-in, complete, cancel (reason required), no-show (only after start time).

Owner

US-20 Services — CRUD; cannot delete a service with future bookings (deactivate instead).

US-21 Staff — CRUD, skills, weekly hours, time off. Adding time off over existing bookings shows the conflicting bookings to reassign.

US-22 Settings — all settings listed in §3, with defaults.

Staff

US-30 My schedule — sees only own appointments; today first.

6. Non-functional requirements

Area	Requirement
Performance	Slot lookup < 500 ms for 1 day, 10 staff
Availability	99.5% monthly; daily DB backups, 14-day retention
Security	HTTPS, hashed passwords, role-based access, rate-limited OTP and booking endpoints, CSRF, audit log for booking changes
Privacy	Store only necessary customer data; customer phone visible only to salon users
Devices	Booking portal works on mobile Chrome/Safari (last 2 versions); dashboard on desktop/tablet
Language	English UI; text kept in translation files (Sinhala/Tamil later)
Timezone	Asia/Colombo for display and rules
Currency	LKR, stored in cents

7. Assumptions & open questions

See `docs/11-open-questions.md`. Phase 1 build can start; unanswered items use the defaults in `docs/02-business-rules.md`.

8. Sign-off

Name	Role	Signature	Date
	KV Saloon		
	Webxkey		

SECTION 2

Business Rules: Booking & Availability

This is the highest-priority document. Code and tests must match these rules exactly. Every rule has an ID (e.g. `BR-4.2`); reference it in tests and PRs.

1. Settings & defaults

All settings are per salon (`salon_settings`). Defaults apply until the owner changes them.

Key	Default	Meaning
`timezone`	`Asia/Colombo`	All rules evaluated in this timezone
`slot_interval_minutes`	15	Gap between offered start times
`lead_time_minutes`	60	Earliest online booking = now + this
`booking_window_days`	30	Latest online booking date
`cancellation_window_minutes`	180	Customer can self-cancel/reschedule until start − this
`assignment_strategy`	`least_booked_today`	`least_booked_today` | `round_robin` | `priority`
`otp_required`	true	Require SMS OTP for online bookings
`reminder_offsets_minutes`	[1440, 120]	Reminders 24h and 2h before
`max_services_per_booking`	4	Online limit
`allow_staff_status_update`	true	Staff can check-in/complete their own items

2. Services

- **BR-2.1** A service has `duration_minutes` (> 0, multiple of 5) and `buffer_after_minutes` (≥ 0).
- **BR-2.2** Staff are blocked for `duration + buffer`; the customer sees only `duration`.
- **BR-2.3** A staff member can perform a service only if a `staff_service` (skill) row exists. A skill may override duration (e.g. junior stylist takes 45 min instead of 30). Effective duration = override ?? service duration.
- **BR-2.4** Inactive or non-online services are hidden online but usable by front desk (non-online only).

3. Staff working time

- **BR-3.1** Working time for a date = weekly schedule for that weekday (one or more intervals, e.g. 09:00–13:00, 14:00–19:00) **unless** a date override exists for that staff and date (override replaces the weekly schedule; an override with no intervals = day off).
- **BR-3.2** Time off (`staff_time_off`, start–end datetime) is subtracted from working time.
- **BR-3.3** Salon closures (`salon_closures`, full day or interval) are subtracted for all staff.
- **BR-3.4** Working time is clipped to salon opening hours for that weekday.
- **BR-3.5** Staff with `is_active = false` have no working time. Staff with `bookable_online = false` are excluded only from online booking.

4. Availability (slot generation)

Input: salon, date, list of services (ordered), staff choice (specific id or `any`).

- **BR-4.1 Sequential services.** Multiple services in one booking are performed back to back **by the same staff member** in Phase 1. Total block = Σ(effective duration) + buffer of the **last** service only. (Buffer between consecutive services of the same booking is not applied.)
- **BR-4.2 Eligible staff** = active, (online: bookable_online), skilled for **every** selected service.
- **BR-4.3 Free time** for a staff on a date = working time (§3) minus existing appointment items whose status blocks time (§7), where each existing item blocks `[start, end + its buffer)`.
- **BR-4.4 Candidate starts** are generated from the beginning of each free interval, aligned to the slot grid (`00:00 + n × slot_interval`, in salon time). A start is valid if `[start, start + total block)` fits fully inside one free interval.
- **BR-4.5 Online limits:** start ≥ now + lead time; date ≤ today + booking window.
- **BR-4.6 Specific staff:** return that staff's valid starts.
- **BR-4.7 Any staff:** return the union of valid starts of all eligible staff. Do not reveal which staff are free.
- **BR-4.8** The availability endpoint is read-only and never reserves a slot. Final validation happens on booking (§6).

5. Automatic assignment ("Any staff")

At booking time, compute the eligible staff who are free for the chosen start (§4). If none → 409 `SLOT_TAKEN`. Otherwise pick one by strategy:

- **BR-5.1** **`least_booked_today`** (default): fewest booked minutes on that date; tie → staff `display_order`; tie → lowest id.
- **BR-5.2** **`round_robin`:** staff whose `last_assigned_at` is oldest (null first); update `last_assigned_at` on assignment.
- **BR-5.3** **`priority`:** lowest `display_order` first.
- **BR-5.4** Record `assignment_method = auto` and the strategy used on the appointment. Front desk can reassign later (`assignment_method = manual`).
- **BR-5.5** Assignment must be deterministic for the same data (tests rely on this).

6. Creating a booking (concurrency)

All booking writes go through `BookingService::create()`:

1. Validate input and settings limits.
2. Open DB transaction.
3. Lock the candidate staff rows: `SELECT ... FROM staff WHERE id IN (...) ORDER BY id FOR UPDATE` (ordered to avoid deadlocks).
4. Re-run the availability check for the requested start **inside** the transaction (it now sees committed bookings).
5. If "Any staff", choose staff by §5 from those still free.
6. Insert `appointments` + `appointment_items`.
7. Commit. Dispatch notifications **after commit** (`afterCommit`).
8. If no staff is free → roll back, return HTTP 409 `SLOT_TAKEN`.

- **BR-6.1** Two concurrent requests for the same staff and overlapping time: exactly one succeeds. (Required test.)
- **BR-6.2** Online bookings are created with status `confirmed` (no payment in Phase 1).

- **BR-6.3** Reference code: `KV-` + 6 chars from `ABCDEFGHJKLMNPQRSTUVWXYZ23456789`, unique per salon.
- **BR-6.4** Customer is matched by normalised phone (E.164 `+94…`); create if not found; update name if empty.
- **BR-6.5** Rate limits: availability 60/min per IP; booking create 5/10 min per phone and 10/10 min per IP; OTP send 3/15 min per phone.
- **BR-6.6 Front desk override:** owner/receptionist may set `force = true` to book outside working time or over an overlap; requires a reason; logged in `audit_logs`. Still never allowed if staff is inactive.

7. Appointment statuses

```
confirmed ──► checked_in ──► completed
    │              │
    ├──► cancelled  └──► cancelled (front desk only)
    └──► no_show  (only after start time)
```

- **BR-7.1** Statuses that block time: `confirmed`, `checked_in`, `completed`. `cancelled` and `no_show` free the time.
- **BR-7.2** Cancellation stores `cancelled_by` (customer/user id), `cancel_reason`, `cancelled_at`.
- **BR-7.3** Customer self-cancel/reschedule only when `now < start − cancellation_window`.
- **BR-7.4** Reschedule = validate new time via §6 with the same services; keeps the same reference; if "Any staff" originally, re-assign; logs old values.
- **BR-7.5** `completed` and `no_show` are final in Phase 1 (owner can reopen with audit log).

8. Notifications

- **BR-8.1** Confirmation on create; notice on reschedule and cancel; reminders at each `reminder_offsets_minutes` before start, only if status is still `confirmed`.
- **BR-8.2** Reminders are scheduled by a job that runs every 5 minutes and sends reminders due in the next window; each reminder is sent at most once (`notification_logs` unique on appointment + type + offset).
- **BR-8.3** If a booking is created less than an offset before start, that reminder is skipped.
- **BR-8.4** SMS text ≤ 160 GSM characters where possible; includes salon name, date, time, reference, manage link (shortened).
- **BR-8.5** Notification failures never fail the booking; they are retried 3 times with backoff and logged.

9. Staff time off over existing bookings

- **BR-9.1** Adding time off that overlaps bookings is allowed but returns the conflicting appointments; the UI must prompt to reassign or cancel them.

10. Worked example

Salon open 09:00–19:00. Slot interval 15. Service "Haircut" 30 min, buffer 10. Staff A works 09:00–13:00; has a booking 09:30–10:00 (+10 buffer → blocked to 10:10).

- Free intervals: 09:00–09:30, 10:10–13:00.
- Block needed: 40 min.

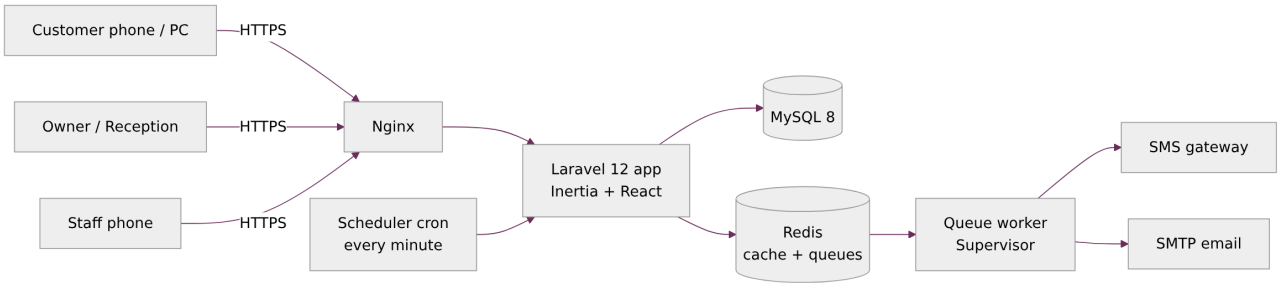
- 09:00 invalid (09:00–09:40 exceeds 09:30). Grid starts after 10:10: 10:15, 10:30 … last valid 12:15 (12:15–12:55).
- Offered: 10:15, 10:30, …, 12:15.

SECTION 3

Architecture & Project Structure

1. Overview

A single Laravel application (modular monolith) serving three front ends from one codebase:



Why a monolith: one small team, one salon, fast delivery. Domain modules keep it clean so later phases (POS, payroll, inventory) plug in without a rewrite.

2. Stack

Layer	Choice	Notes
Language	PHP 8.3	strict_types in all files
Framework	Laravel 12	Sanctum not needed in Phase 1 (session auth)
Front end	Inertia.js 2 + React 18 + TypeScript	One SPA-like app; routes stay in Laravel
UI	Tailwind CSS + shadcn/ui components	Calendar: FullCalendar (resource day view)
DB	MySQL 8, InnoDB, utf8mb4	Row locks for booking
Cache/ Queue	Redis	`QUEUE_CONNECTION=redis`
Auth	Laravel Breeze (Inertia React)	Roles via `spatie/laravel-permission`
Tests	Pest, Laravel factories; Vitest	Playwright E2E for booking flow (BK-803)
Quality	Pint, Larastan level 6, ESLint, Prettier	Run in CI
CI	GitHub Actions	test + lint on every PR
Hosting	Webxkey Azure VM, Ubuntu, Nginx, PHP-FPM, Supervisor	Let's Encrypt SSL

3. Folder structure

```
app/
  Domain/
    Booking/
      Actions/          CreateBooking, RescheduleBooking, CancelBooking, ChangeStatus
      Services/         AvailabilityService, AssignmentService, BookingService
      Strategies/       LeastBookedToday, RoundRobin, Priority (implement AssignmentStrategy)
      DTOs/             SlotQuery, BookingRequestData, Slot
      Enums/            AppointmentStatus, AssignmentMethod, BookingSource
      Exceptions/       SlotTakenException, OutsideCancellationWindowException
      Events/           BookingCreated, BookingRescheduled, BookingCancelled
    Catalog/            Service, ServiceCategory logic
    Staff/              Schedules, time off, skills
    Customers/          Customer matching (phone normalisation)
    Notifications/      SmsChannel, SmsGateway interface, drivers (NotifyLk, TextLk, Log)
    Salon/              Settings, closures, opening hours
    Shared/             TimeRange, Interval maths, Money, PhoneNumber
  Http/
    Controllers/
      Public/           BookingPortalController, ManageBookingController
      Api/Public/       AvailabilityController, BookingController, OtpController
      Admin/            CalendarController, AppointmentController, ServiceController,
StaffController, SettingsController
      Staff/            MyScheduleController
    Requests/           Form requests (validation only)
    Resources/          API resources (JSON shape)
  Models/               Eloquent models (thin, relations + casts + scopes)
  Policies/
  Jobs/                 SendReminders, SendSms
database/
  migrations/ factories/ seeders/
resources/js/
  Pages/
    Booking/            Services, Staff, Slots, Details, Confirmation, Manage
    Admin/              Calendar, Appointments, Services, Staff, Settings, Users
    Staff/              Today
  Components/ Layouts/ lib/ types/
routes/
  web.php  admin.php  staff.php  api.php
tests/
  Unit/Domain/Booking/  Feature/  Browser/
docs/
```

4. Key design rules

- **Controllers are thin:** validate (FormRequest) → call one Action/Service → return Inertia page or Resource.
- **Domain services are pure where possible.** `AvailabilityService` takes data (schedules, bookings, settings, now) and returns slots; DB loading is done by a separate `AvailabilityRepository`. This makes the rules unit-testable without a database.
- **Interval maths** lives in `Domain/Shared/TimeRange` (subtract, intersect, merge, fits). Fully unit-tested.
- **Clock:** inject `Psr\Clock\ClockInterface` / use `Carbon::setTestNow()` in tests; never call `now()` inside domain services directly.

- **Tenancy:** a `BelongsToSalon` trait adds a global scope on `salon_id`; current salon resolved by middleware (Phase 1: single salon from config `SALON_ID`).
- **Events → listeners** for notifications, dispatched `afterCommit`.
- **SMS gateway** behind `SmsGateway` interface; `log` driver for local/dev; real driver chosen by `SMS_DRIVER`.

5. Routes (summary)

Area	Prefix	Auth
Booking portal	`/` and `/book/*`	Public
Manage booking	`/b/{reference}?signature=…`	Signed URL
Public API	`/api/v1/public/*`	Public + rate limits
Admin	`/admin/*`	`auth`, role owner/receptionist
Staff panel	`/staff/*`	`auth`, role staff

6. Environments

Env	URL (example)	Data	Notes
Local	`http://localhost:8000`	Seeded demo	SMS driver = log
Staging	`staging-kv.webxkey.com`	Seeded demo + client test	Client UAT; SMS to test numbers only
Production	`book.kvsaloon.lk` (TBC)	Real	Daily backup, monitoring

See `docs/10-deployment.md`.

7. Future phases fit

Phase	New module	Hooks into
2 Customers	`Domain/Customers` (profiles, history)	`customers`, `appointments`
3 Staff & Payroll	`Domain/Payroll`	`staff`, completed `appointment_items` (commission base)
4 Inventory/ Reports	`Domain/Inventory`, `Domain/Reports`	items, expenses

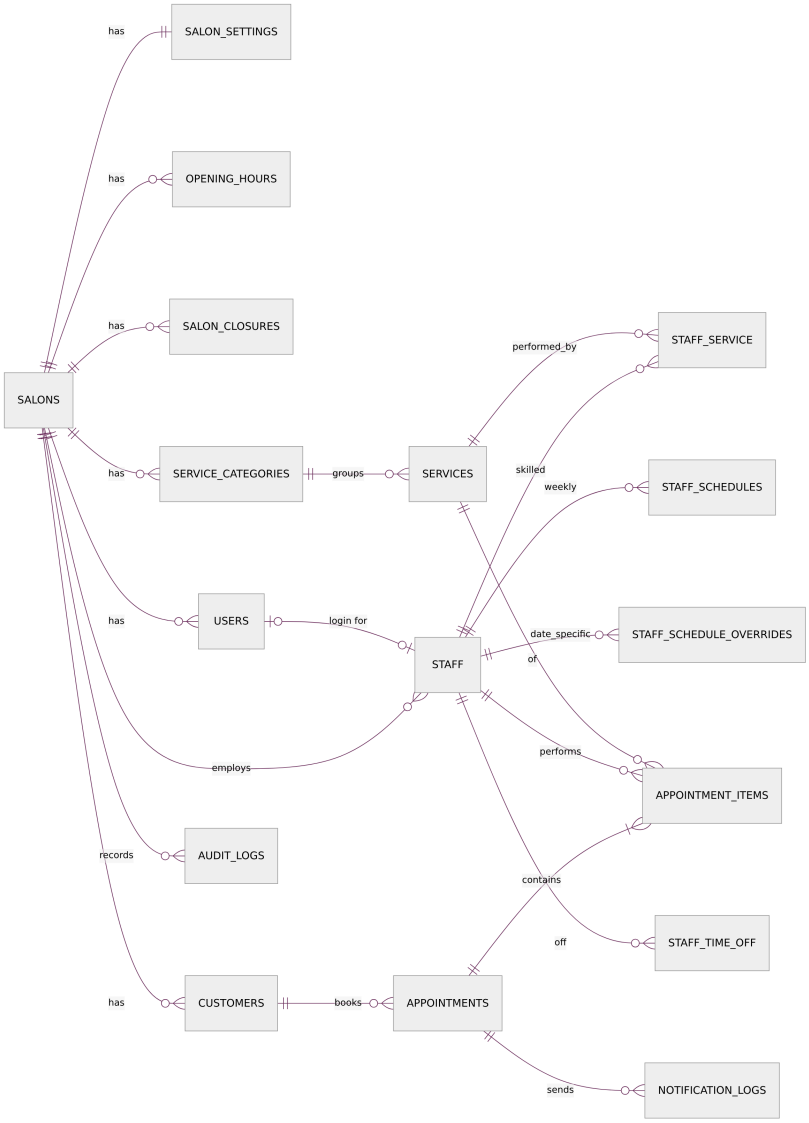
Keep `appointment_items.price_cents` as the price at booking time — Phase 3 commission depends on it.

SECTION 4

Database Schema (Phase 1)

MySQL 8, InnoDB, `utf8mb4_unicode_ci`. All tables have `id` (bigint unsigned PK) and `created_at` / `updated_at` unless noted. Datetimes are **UTC** (`datetime`, not `timestamp`). Money = integer cents.

1. ERD



2. Tables

salons

Column	Type	Notes
name	varchar(120)	"KV Saloon"
slug	varchar(60) unique	

Column	Type	Notes
phone, email, address	varchar	shown in notifications
logo_path	varchar null	
is_active	bool default 1	

salon_settings

`salon_id` unique FK, plus one column per setting in `02-business-rules.md §1` (`timezone`, `slot_interval_minutes` smallint, `lead_time_minutes`, `booking_window_days`, `cancellation_window_minutes`, `assignment_strategy` enum, `otp_required` bool, `reminder_offsets_minutes` json, `max_services_per_booking`, `allow_staff_status_update` bool).

opening_hours

Column	Type	Notes
salon_id	FK	
weekday	tinyint	0 = Sunday … 6 = Saturday (ISO alt not used)
opens_at, closes_at	time	salon local time; no row = closed
Unique (`salon_id`, `weekday`, `opens_at`).		

salon_closures

`salon_id`, `starts_at` datetime, `ends_at` datetime, `reason` varchar. Index (`salon_id`, `starts_at`).

users

Laravel default + `salon_id` FK null (null = Webxkey super admin), `phone` varchar null, `is_active` bool. Roles via spatie tables: `owner`, `receptionist`, `staff`, `super_admin`.

service_categories

`salon_id`, `name`, `sort_order` smallint.

services

Column	Type	Notes
salon_id	FK	
category_id	FK service_categories	
name	varchar(120)	
description	text null	
duration_minutes	smallint	> 0, multiple of 5
buffer_after_minutes	smallint default 0	
price_cents	int unsigned	LKR cents
price_is_from	bool default 0	shows "from LKR …"
is_active	bool default 1	

Column	Type	Notes
bookable_online	bool default 1	
sort_order	smallint	
deleted_at	datetime null	soft delete

staff

Column	Type	Notes
salon_id	FK	
user_id	FK users null unique	login account
name, display_name	varchar	display_name shown to customers
phone, email	varchar null	
photo_path	varchar null	
bio	varchar(300) null	
display_order	smallint	used by priority strategy
is_active	bool	
bookable_online	bool	
last_assigned_at	datetime null	round-robin
deleted_at	datetime null	

staff_service (skills)

`staff_id`, `service_id`, `duration_override_minutes` smallint null. PK (`staff_id`, `service_id`).

staff_schedules (weekly template)

`staff_id`, `weekday` tinyint, `starts_at` time, `ends_at` time. Multiple rows per weekday allowed (split shifts / breaks). Index (`staff_id`, `weekday`).

staff_schedule_overrides

`staff_id`, `date` date, `starts_at` time null, `ends_at` time null. A row with null times = day off. Multiple rows per date allowed. Index (`staff_id`, `date`).

staff_time_off

`staff_id`, `starts_at` datetime, `ends_at` datetime, `type` enum(`leave`,`sick`,`break`,`other`), `note` varchar null, `created_by` FK users. Index (`staff_id`, `starts_at`, `ends_at`).

customers

Column	Type	Notes
salon_id	FK	
name	varchar(120)	
phone_e164	varchar(16)	`+94771234567`

Column	Type	Notes
email	varchar null	
notes	text null	internal (e.g. allergies)
marketing_opt_in	bool default 0	
phone_verified_at	datetime null	
Unique (`salon_id` , `phone_e164`).		

appointments

Column	Type	Notes
salon_id	FK	
reference	char(9)	`KV-XXXXXX` ; unique (`salon_id` , `reference`)
customer_id	FK	
status	enum	`confirmed` , `checked_in` , `completed` , `cancelled` , `no_show`
source	enum	`online` , `front_desk` , `phone` , `walk_in`
assignment_method	enum	`auto` , `customer_choice` , `manual`
assignment_strategy	varchar null	strategy used when auto
starts_at, ends_at	datetime	of the whole visit (UTC)
total_price_cents	int unsigned	snapshot
customer_note	varchar(500) null	
internal_note	text null	
cancelled_at	datetime null	
cancelled_by_user_id	FK null	null + cancelled = customer
cancel_reason	varchar null	
forced	bool default 0	front desk override
created_by_user_id	FK null	null = online
deleted_at	datetime null	
Indexes: (`salon_id` , `starts_at`), (`customer_id`), (`status`).		

appointment_items

Column	Type	Notes
appointment_id	FK cascade	
service_id	FK	
staff_id	FK	
starts_at, ends_at	datetime	ends_at = start + effective duration
buffer_after_minutes	smallint	snapshot; staff blocked to ends_at + buffer

Column	Type	Notes
price_cents	int unsigned	snapshot (Phase 3 commission base)
status	enum	mirrors appointment in Phase 1
sequence	tinyint	order within appointment
Index (`staff_id`, `starts_at`, `ends_at`) — **critical for availability queries**.		

otp_codes

`phone_e164`, `code_hash`, `expires_at`, `attempts` tinyint, `verified_at` null, `ip`. Index (`phone_e164`, `expires_at`). Codes: 6 digits, 5-minute expiry, max 5 attempts.

notification_logs

`appointment_id` FK, `channel` enum(`sms`,`email`), `type` enum(`confirmation`,`reminder`,`rescheduled`,`cancelled`,`staff_alert`), `offset_minutes` int null, `to`, `status` enum(`queued`,`sent`,`failed`), `provider_message_id`, `error` text null, `sent_at`. Unique (`appointment_id`, `channel`, `type`, `offset_minutes`) to prevent duplicates.

audit_logs

`salon_id`, `user_id` null, `action` varchar (`appointment.created`, `appointment.moved`, `appointment.forced`, …), `subject_type`, `subject_id`, `old_values` json, `new_values` json, `ip`, `created_at`.

3. Overlap rule (reference SQL)

Existing items that block staff `:staff` for a range `[:start, :end)`:

```
SELECT ai.id FROM appointment_items ai
JOIN appointments a ON a.id = ai.appointment_id
WHERE ai.staff_id = :staff
  AND a.status IN ('confirmed','checked_in','completed')
  AND a.deleted_at IS NULL
  AND ai.starts_at < :end
  AND DATE_ADD(ai.ends_at, INTERVAL ai.buffer_after_minutes MINUTE) > :start;
```

4. Seed data (local & staging)

- Salon "KV Saloon", opening hours Mon–Sat 09:00–19:00, Sun 10:00–16:00
- 4 staff: Nimal (all hair), Kavindi (hair + colour), Shanika (beauty, nails), Ruwan (hair, beard) with different schedules, one with a lunch break
- ~12 services across Hair, Colour, Beauty, Nails
- Users: owner@kv.test, reception@kv.test, nimal@kv.test (password `password`, local only)
- ~30 random appointments over the next 7 days

SECTION 5

API Specification (Phase 1)

Admin and staff screens use **Inertia** (server-rendered props), so they do not need a JSON API. The **public booking flow** uses a small JSON API so the booking pages can refresh slots without full reloads, and so a future mobile app can reuse it.

Base: `/api/v1/public`. JSON only. Times in responses are ISO 8601 **with salon offset** (`2026-11-03T10:15:00+05:30`). Money: `{ "cents": 250000, "formatted": "LKR 2,500.00" }`.

Error format

```
{ "error": { "code": "SLOT_TAKEN", "message": "This time was just booked. Please choose another.", "fields": {} } }
```

HTTP	code	When
422	`VALIDATION_FAILED`	Bad input; `fields` holds per-field messages
409	`SLOT_TAKEN`	No eligible staff free at booking time
403	`OTP_REQUIRED` / `OTP_INVALID`	OTP missing or wrong
403	`OUTSIDE_CANCELLATION_WINDOW`	Too late to change
404	`NOT_FOUND`	
429	`RATE_LIMITED`	See BR-6.5

GET `/salon`

Public salon info and booking settings.

```
{ "name": "KV Saloon", "phone": "+94...", "address": "...", "timezone": "Asia/Colombo",
  "booking": { "lead_time_minutes": 60, "booking_window_days": 30, "max_services": 4,
"otp_required": true } }
```

GET `/services`

Active, online-bookable services grouped by category.

```
{ "categories": [ { "id": 1, "name": "Hair", "services": [
  { "id": 3, "name": "Haircut", "description": null, "duration_minutes": 30,
    "price": { "cents": 150000, "formatted": "LKR 1,500.00" }, "price_is_from": false } ] } ] }
```

GET `/staff?service_ids[]=3&service_ids[]=7`

Staff eligible for **all** given services (BR-4.2).

```
{ "data": [ { "id": 2, "display_name": "Kavindi", "photo_url": "...", "bio": "..." } ] }
```

GET `/availability/dates?service_ids[]=3&staff_id=any&from=2026-11-01&to=2026-11-30`

Dates with at least one slot. `staff_id` = id or `any`.

```
{ "dates": [ { "date": "2026-11-03", "available": true }, { "date": "2026-11-04", "available":
false } ] }
```

GET `/availability/slots?service_ids[]=3&staff_id=any&date=2026-11-03`

```
{ "date": "2026-11-03", "duration_minutes": 30, "generated_at": "2026-10-29T13:00:00+05:30",
  "slots": [ { "start": "2026-11-03T10:15:00+05:30", "end": "2026-11-03T10:45:00+05:30" } ] }
```

`end` = customer-visible end (no buffer). Never includes staff ids for `any`.

POST `/otp/send`

```
{ "phone": "0771234567" }
```

→ `204`. Phone normalised to `+94771234567`. Rate-limited.

POST `/otp/verify`

```
{ "phone": "0771234567", "code": "482913" }
```

→ `200 { "otp_token": "..." }` (short-lived signed token, 15 min, tied to phone).

POST `/bookings`

Header: `Idempotency-Key: <uuid>` (required; same key within 10 min returns the original result).

```
{ "service_ids": [3, 7], "staff_id": "any", "start": "2026-11-03T10:15:00+05:30",
  "customer": { "name": "Dilani Perera", "phone": "0771234567", "email": null },
  "note": "Short layers please", "consent": true, "otp_token": "..." }
```

→ `201`

```
{ "reference": "KV-7H3K9Q", "status": "confirmed",
  "start": "2026-11-03T10:15:00+05:30", "end": "2026-11-03T11:30:00+05:30",
  "staff": { "id": 2, "display_name": "Kavindi" },
  "items": [ { "service": "Haircut", "start": "...", "end": "..." }, { "service": "Blow dry",
"start": "...", "end": "..." } ],
  "total": { "cents": 350000, "formatted": "LKR 3,500.00" },
  "manage_url": "https://book.kvsaloon.lk/b/KV-7H3K9Q?signature=..." }
```

Manage booking (signed)

All require the `signature` query param from `manage_url`.

Method	Path	Body	Result
GET	`/bookings/{reference}`	—	Booking (same shape as create) + `can_change` bool + `change_deadline`
GET	`/bookings/{reference}/slots?date=`	—	Slots for same services and staff rule, excluding own booking
POST	`/bookings/{reference}/reschedule`	`{ "start": "..." }`	200 booking, or 409 / 403
POST	`/bookings/{reference}/cancel`	`{ "reason": "optional" }`	200 booking, or 403

Admin actions (Inertia routes, for reference)

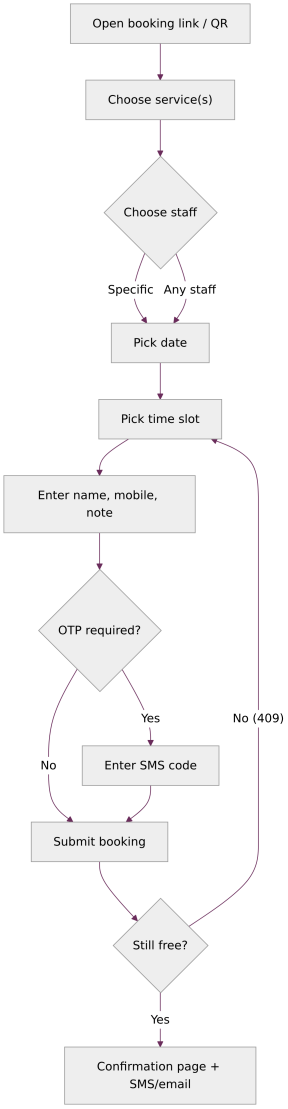
Method	Path	Purpose
GET	`/admin/calendar?date=&view=day`	Calendar page props: staff, items, time off
GET	`/admin/calendar/events?from=&to=`	JSON feed for FullCalendar (JSON response)
POST	`/admin/appointments`	Create (supports `force` + `force_reason`)
PATCH	`/admin/appointments/{id}/move`	`{ start, staff_id, force? }`
PATCH	`/admin/appointments/{id}/status`	`{ status, reason? }`
GET/POST/PATCH/DELETE	`/admin/services`, `/admin/service-categories`	CRUD
GET/POST/PATCH/DELETE	`/admin/staff` (+ `/schedule`, `/time-off`, `/skills`)	CRUD
GET/PATCH	`/admin/settings`	Salon settings, opening hours, closures
GET	`/admin/availability/slots`	Same as public, but includes non-online services/staff and staff ids
GET	`/staff/today`	Staff panel
PATCH	`/staff/items/{id}/status`	Staff check-in / complete own item

All admin writes go through the same `BookingService` as the public API.

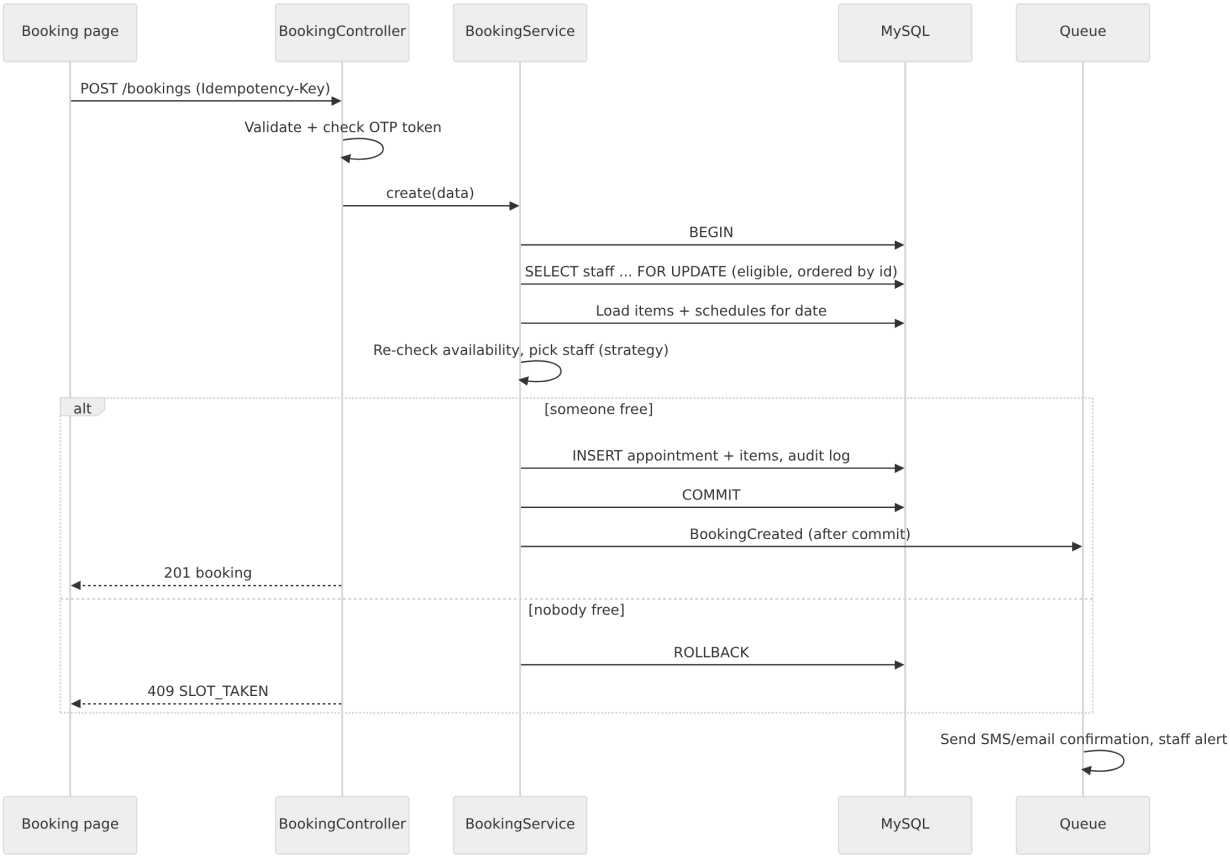
SECTION 6

User Flows & Screens

1. Customer online booking flow



2. Booking creation (server sequence)



3. Front desk walk-in flow

Calendar → click empty cell (staff + time prefilled) → search customer by phone (or create) → choose services → save. If the cell does not fit, show the conflict and offer "Force book" (reason required).

4. Reminder flow

Scheduler every 5 min → `SendReminders` job finds confirmed appointments whose `starts_at − offset` is within the last 5 min and not yet logged → queue SMS → write `notification_logs`.

5. Screen inventory

#	Screen	Route	Role	Key elements
P1	Service selection	`/book`	Public	Categories, service cards, multi-select, sticky summary (total time, price)
P2	Staff selection	`/book/staff`	Public	"Any staff" card first, staff cards with photo
P3	Date & time	`/book/time`	Public	Horizontal date strip (14 days, scroll), slots grouped Morning / Afternoon / Evening
P4	Your details	`/book/details`	Public	Name, mobile, email (opt), note, consent
P5	OTP	`/book/verify`	Public	6-digit input, resend timer
P6	Confirmation	`/book/done/{ref}`	Public	

#	Screen	Route	Role	Key elements
				Reference, summary, add-to-calendar (.ics), manage link
P7	Manage booking	`/b/{ref}`	Signed	Details, reschedule, cancel, salon phone
A1	Login	`/login`	Users	
A2	Calendar	`/admin/calendar`	Owner, Reception	Day/week, staff columns, drag/drop, status colours, "+ New"
A3	Appointment drawer	(panel)	Owner, Reception	Details, status buttons, move, cancel, notes, history
A4	New booking dialog	(dialog)	Owner, Reception	Customer search, services, staff, time, force
A5	Appointments list	`/admin/appointments`	Owner, Reception	Filters: date, staff, status, source; search
A6	Services	`/admin/services`	Owner	Categories, drag sort, edit form
A7	Staff list & profile	`/admin/staff`	Owner	Profile, skills (checkbox list + duration override), weekly schedule editor, time off
A8	Settings	`/admin/settings`	Owner	Booking rules, opening hours, closures, notifications
A9	Users	`/admin/users`	Owner	Invite, role, deactivate
S1	My day	`/staff/today`	Staff	Timeline of own items, check-in / complete

6. UI guidelines

- Mobile-first public pages (design at 375 px wide), large tap targets (≥ 44 px)
- Salon branding: logo and accent colour from settings (KV Saloon to supply)
- Status colours: confirmed = blue, checked-in = amber, completed = green, cancelled = grey (hidden by default), no-show = red
- Always show times in salon time with am/pm (e.g. "10:15 am")
- Loading skeletons on slot lists; friendly empty state ("No times left on this day — try another date")
- Wireframes: to be produced in Figma/Canva before BK-501 starts (owner: Webxkey)

SECTION 7

Roadmap & Task Backlog (Phase 1)

Each task is sized for **one AI-agent session + one PR** (roughly half a day to two days). Work strictly in dependency order. Copy a task block into the agent prompt (see `docs/12-ai-agent-playbook.md`).

Status legend: ☐ todo · ◐ in progress · ☑ done

Milestones

Milestone	Tasks	Target (weeks from start)	Demo to client
M0 Foundation	BK-001 – BK-004	Week 1	—
M1 Data & Catalog admin	BK-101 – BK-104	Week 2	Services & staff setup
M2 Availability engine	BK-201 – BK-204	Week 3	Slot list for test dates
M3 Booking core	BK-301 – BK-304	Week 4	Book via API, no double booking
M4 Customer portal	BK-401 – BK-404	Week 5	Full online booking on phone
M5 Front desk & staff	BK-501 – BK-504	Week 5–6	Calendar, walk-ins, staff view
M6 Notifications	BK-601 – BK-603	Week 6	SMS confirmations & reminders
M7 Launch	BK-701 – BK-704	Week 7	UAT, go-live

M0 Foundation

BK-001 Project scaffold ☐

- Read first: AGENTS.md, 03-architecture
- Do: Laravel 12 + Breeze (Inertia React TS), Tailwind, Pest, Pint, Larastan, ESLint/Prettier, `app/Domain` folders, `.env.example`, `README` run steps.
- Accept: `php artisan test` and `npm run build` pass; folder tree matches §3 of architecture.

BK-002 CI pipeline ☐ — Depends: BK-001

- GitHub Actions: MySQL + Redis services, composer/npm cache, run Pint, Larastan, Pest, typecheck, lint on PRs.
- Accept: a failing test blocks merge.

BK-003 Roles & auth ☐ — Depends: BK-001

- spatie/laravel-permission; roles owner, receptionist, staff, super_admin; route groups `/admin`, `/staff` with middleware; policies skeleton.
- Accept: feature tests prove each role can/cannot reach each area.

BK-004 Shared value objects ☐ — Depends: BK-001

- `TimeRange` (intersect, subtract, merge, contains, fits), `Money`, `PhoneNumber` (Sri Lanka normalisation to E.164), salon `Clock`.
- Accept: ≥ 95% unit coverage on these classes; edge cases (touching ranges, empty results, midnight).

M1 Data & Catalog admin

BK-101 Migrations & models ☐ — Depends: BK-004 — Read: 04-database-schema

- All Phase 1 tables, models, relations, casts, enums, `BelongsToSalon` scope, factories.
- Accept: `migrate:fresh` works; factories create valid graphs; schema matches doc.

BK-102 Seeder ☐ — Depends: BK-101

- Demo data exactly as schema §4.
- Accept: `migrate --seed` gives a usable demo.

BK-103 Services admin ☐ — Depends: BK-101, BK-003 — Read: PRD US-20

- Categories + services CRUD, sort, deactivate; cannot delete with future bookings.
- Accept: feature tests for validation and the delete rule.

BK-104 Staff admin ☐ — Depends: BK-103 — Read: PRD US-21, BR-3

- Staff CRUD, photo upload, skills with duration override, weekly schedule editor (multiple intervals/day), date overrides, time off (with BR-9.1 conflict list), link to user account.
- Accept: feature tests; schedule editor rejects overlapping intervals.

M2 Availability engine

BK-201 Working-time calculator ☐ — Depends: BK-101 — Read: BR-3

- Pure service: (date, staff schedule, overrides, time off, closures, opening hours) → list of TimeRange in UTC.
- Accept: unit tests for each BR-3.x rule incl. split shifts and overrides.

BK-202 Slot generator ☐ — Depends: BK-201 — Read: BR-2, BR-4

- Pure `AvailabilityService::slots(SlotQuery)`; supports specific staff and any; multi-service totals; grid alignment; lead time and window.
- Accept: unit test reproducing the worked example in BR-10 exactly; tests per BR-4.x.

BK-203 Availability repository + endpoints ☐ — Depends: BK-202 — Read: API spec

- Efficient loading (≤ 5 queries per day request), `/services`, `/staff`, `/availability/dates`, `/availability/slots`, rate limits.
- Accept: feature tests; response shapes match spec; 30-day dates request < 1 s on seed data.

BK-204 Assignment strategies ☐ — Depends: BK-202 — Read: BR-5

- `AssignmentStrategy` interface + 3 implementations, chosen from settings.
- Accept: deterministic unit tests for each strategy and tie-breaks.

M3 Booking core

BK-301 BookingService::create ☐ — Depends: BK-203, BK-204 — Read: BR-6

- Transaction, ordered `FOR UPDATE` locks, re-check, assign, insert, reference code, customer match, audit log, `BookingCreated` after commit.
- Accept: **concurrency test** (two parallel processes, same slot → one 201, one 409); all BR-6 rules tested.

BK-302 Public booking endpoint + OTP ☐ — Depends: BK-301

- `/otp/send`, `/otp/verify`, `POST /bookings` with idempotency key, rate limits, consent.
- Accept: feature tests incl. idempotent replay and rate limits.

BK-303 Reschedule & cancel ☐ — Depends: BK-301 — Read: BR-7

- Actions + signed manage endpoints; cancellation window; audit old values.
- Accept: tests for window edges (exactly at deadline), re-assignment on reschedule.

BK-304 Status transitions ☐ — Depends: BK-301 — Read: BR-7

- State machine with allowed transitions only; no-show only after start.
- Accept: table-driven tests for every from→to pair.

M4 Customer portal

BK-401 Booking pages P1–P3 ☐ — Depends: BK-203 — Read: 06 screens

- Services, staff, date strip, slots; state kept in URL query so refresh works.
- Accept: works at 375 px; Vitest for booking-state helpers.

BK-402 Details, OTP, confirmation P4–P6 ☐ — Depends: BK-302, BK-401

- Handles 409 by returning to slots with message; .ics download.
- Accept: Playwright happy path + 409 path.

BK-403 Manage booking P7 ☐ — Depends: BK-303

BK-404 PWA & branding ☐ — Depends: BK-401

- Manifest, icons, theme colour from settings, offline fallback page, SEO meta, QR code generator for the salon (admin download).

M5 Front desk & staff

BK-501 Calendar view ☐ — Depends: BK-304 — Read: US-10

- FullCalendar resource day/week, events feed, time off shading, status colours, appointment drawer. **BK-502 Create & move from calendar** ☐ — Depends: BK-501

- New booking dialog, customer search by phone, drag/drop move (server-validated), force with reason. **BK-503 Appointments list & search** ☐ — Depends: BK-501

BK-504 Staff panel S1 ☐ — Depends: BK-304

M6 Notifications

BK-601 SMS gateway abstraction ☐ — Depends: BK-001

- `SmsGateway` interface, `log` driver, real driver (Notify.lk or as confirmed), message templates in lang files. **BK-602 Booking notifications** ☐ — Depends: BK-601, BK-301

- Confirmation, reschedule, cancel, staff alert; email (Markdown mail); `notification_logs`; retries. **BK-603 Reminders** ☐ — Depends: BK-602 — Read: BR-8

- Scheduled job, de-duplication, skip rule BR-8.3.
- Accept: tests using `Carbon::setTestNow()` across offsets.

M7 Launch

BK-701 Settings screen ☐ — Depends: BK-104

BK-702 Security & performance pass ☐ — rate limits, headers, policy review, N+1 check, indexes, Larastan clean.

BK-703 Staging deploy & UAT ☐ — Read: 10-deployment, 09-test-plan. Client tests with UAT checklist.

BK-704 Production go-live ☐ — data entry of real services/staff, backups verified, monitoring on, handover & training.

Definition of Done (every task)

- Acceptance criteria met and tested
- CI green; no new Larastan errors
- Docs updated if behaviour/schema changed; CHANGELOG line added
- PR reviewed by a human (Farhan or partner) and merged

SECTION 8

Coding Standards, Git Workflow & Review

1. Git workflow

- `main` = production-ready. `develop` = integration (deployed to staging).
- Branches: `feature/BK-301-booking-service`, `fix/BK-301-lock-order`, `docs/...`.
- One task per branch and PR. Rebase on `develop` before opening the PR.
- Commits: Conventional Commits — `feat(booking): add round-robin strategy [BK-204]`.
- Merge to `develop` by squash; release to `main` via PR with tag `v1.0.0`.
- Protect `main` and `develop`: CI must pass, 1 approval required.

2. PHP / Laravel

- `declare(strict_types=1);` everywhere. Typed properties, return types, readonly DTOs.
- PSR-12 via Pint (Laravel preset). Larastan level 6, no baseline growth.
- Controllers: max ~30 lines per method; no queries beyond simple lookups.
- Validation in FormRequests; authorisation in Policies.
- Domain classes: final by default; constructor injection; no facades inside `app/Domain` except `DB::transaction`.
- Enums are PHP backed enums; never compare raw status strings.
- Dates: `CarbonImmutable`; convert to salon timezone only at boundaries (rules engine input, UI output).
- No `env()` outside config files.
- Queries: eager-load relations; add an index for every new filter column.

3. TypeScript / React

- Strict TS, no `any`. Shared types in `resources/js/types` (generated or hand-kept in sync with Resources).
- Function components + hooks. Page components in `Pages/`, reusable UI in `Components/`.
- Money and time formatting only via `lib/format.ts`.
- No business rules in the UI — the UI shows what the API returns.
- Accessibility: labels on inputs, keyboard-usable dialogs, colour is never the only signal.

4. Testing

Kind	Tool	Must cover
Unit	Pest	`TimeRange`, working time, slot generator, strategies, status machine
Feature	Pest + MySQL	Endpoints, policies, validation, BookingService incl. concurrency
Front end	Vitest	Booking state helpers, formatters
E2E	Playwright	Online booking happy path, 409 path, reschedule, calendar create

Kind	Tool	Must cover
- Name tests after rules: `it('skips 09:00 when block exceeds free interval (BR-4.4)')`.		
- Use `Carbon::setTestNow()`; never depend on the real clock.		
- Coverage target: `app/Domain` ≥ 90%.		

5. Pull request template

```
## Task
BK-___ — title

## What changed
-

## Business rules touched
BR-__

## Tests
- [ ] Unit  - [ ] Feature  - [ ] E2E (if UI flow)

## Screenshots (UI)

## Checklist
- [ ] CI green
- [ ] Docs/CHANGELOG updated
- [ ] No secrets, no edited merged migrations
- [ ] New packages justified: ___
```

6. Review checklist (human reviewer)

- Does it match the business rule text exactly?
- Any query inside a loop? Missing index?
- Times stored UTC, shown in salon time?
- Salon scoping present?
- Could two requests race here?
- Error messages friendly for customers, specific for staff?

7. Architecture Decision Records

Significant choices go in `docs/decisions/ADR-NNN-title.md`:

```
# ADR-NNN: Title
Date / Status (proposed | accepted | superseded)
## Context
## Decision
## Consequences
```

SECTION 9

Test Plan & UAT Checklist (Phase 1)

1. Critical automated tests (must exist before go-live)

ID	Scenario	Expected	Rule
T-01	Worked example (Haircut 30 + 10 buffer, booking 09:30–10:00)	Slots 10:15 … 12:15	BR-10
T-02	Split shift 09–13, 14–19, 60-min service	No slot 12:15; 14:00 valid	BR-3.1, BR-4.4
T-03	Date override marks day off	No slots that day	BR-3.1
T-04	Time off 15:00–16:00	No slot overlapping it	BR-3.2
T-05	Salon closure (public holiday)	No slots for anyone	BR-3.3
T-06	Staff without skill for 2nd service	Not eligible	BR-4.2
T-07	Skill duration override 45 min	Uses 45 min block	BR-2.3
T-08	Two services 30 + 60, buffers 10 + 15	Block = 105 min	BR-4.1
T-09	Lead time 60 min, now 10:05	First slot ≥ 11:05 on grid → 11:15	BR-4.5
T-10	Beyond booking window	Date unavailable	BR-4.5
T-11	Any staff, all busy at 10:00 except one	Slot shown; booking assigns that one	BR-4.7, BR-5
T-12	least_booked_today tie	display_order then id	BR-5.1
T-13	Round robin over 4 bookings	Each staff once	BR-5.2
T-14	**Concurrent double booking** (2 processes)	One 201, one 409	BR-6.1
T-15	Cancelled booking frees time	Slot reappears	BR-7.1
T-16	Customer cancels 3h01m before / 2h59m before	Allowed / 403	BR-7.3
T-17	Reschedule keeps reference, logs old values	✓	BR-7.4
T-18	No-show before start	Rejected	BR-7
T-19	Reminder 24h & 2h sent once each	2 logs, no duplicates on re-run	BR-8.2
T-20	Booking made 1h before start	2h reminder skipped	BR-8.3
T-21	SMS gateway down	Booking succeeds, failure logged, retried	BR-8.5
T-22	Same Idempotency-Key twice	Same response, one booking	API
T-23	Receptionist cannot open settings; staff sees only own items	403	PRD §2
T-24	Phone formats 077…, +9477…, 9477…	Same customer	BR-6.4
T-25	Early-morning slots cross the UTC date boundary (00:00 UTC = 05:30 local)	Slots grouped under the correct local date	Timezone

2. Non-functional checks

- Slot lookup p95 < 500 ms (seed with 10 staff, 2,000 future items)
- Lighthouse mobile ≥ 85 performance, ≥ 95 accessibility on booking pages
- Security: rate limits verified, signed URLs cannot be altered, no customer phone leaked in public responses, HTTPS + security headers

3. UAT checklist (KV Saloon staff on staging)

Tick each and note issues.

Customer (use your own phone)

☐ Find and open the booking page from the QR code
☐ Book one service with "Any staff"
☐ Book two services with a specific stylist
☐ Receive SMS confirmation; open the manage link
☐ Reschedule to another day; cancel a booking
☐ Try to book a time someone else just took → friendly message

Reception

☐ See today's calendar with all staff
☐ Add a walk-in in under 30 seconds
☐ Move a booking to another stylist by dragging
☐ Check-in, complete, mark no-show
☐ Search a booking by phone number

Owner

☐ Add a new service and price; hide one from online
☐ Add a staff member, set skills and weekly hours
☐ Add leave for a staff member and handle conflicting bookings
☐ Add a holiday closure
☐ Change the assignment rule and reminder times

Staff

☐ Log in on phone, see own day only
☐ Get a new booking alert

UAT sign-off: ____ (KV Saloon) Date: __

4. Defect severity

Severity	Example	Go-live?
Critical	Double booking, wrong times, data loss	Blocks
High	Booking fails for a service, SMS not sent	Blocks
Medium	Layout issue on one device	Fix within 1 week
Low	Wording, colours	Backlog

SECTION 10

Deployment & Operations

1. Server (Webxkey Azure VM)

- Ubuntu LTS, Nginx, PHP 8.3-FPM (+ intl, bcmath, redis, gd), MySQL 8, Redis, Supervisor, Certbot
- One Linux user per app (`kvsaloon`), app path `/var/www/kv-saloon`
- Separate MySQL database + user per environment (`kv_staging`, `kv_prod`), least privilege
- Firewall: 22 (key-only SSH, restricted IPs), 80, 443

2. Processes

Process	How
Web	Nginx → PHP-FPM pool `kvsaloon`
Queue worker	Supervisor: `php artisan queue:work redis --tries=3 --backoff=60 --max-time=3600` (2 workers)
Scheduler	cron: `* * * * * php /var/www/kv-saloon/current/artisan schedule:run`

3. Deployment (zero-downtime)

Use a release-folder deploy (Laravel Envoy, Deployer, or a GitHub Actions SSH script):

1. Clone tag into `releases/<timestamp>`
2. Link shared `.env` and `storage/`
3. `composer install --no-dev --optimize-autoloader`
4. `npm ci && npm run build`
5. `php artisan migrate --force`
6. `php artisan optimize` (config, route, view cache)
7. Switch `current` symlink → reload PHP-FPM → `php artisan queue:restart`
8. Keep last 5 releases; rollback = switch symlink back

Triggers: merge to `develop` → staging; tag `v*` on `main` → production (manual approval).

4. Configuration (.env keys)

```
APP_NAME="KV Saloon" APP_ENV APP_KEY APP_URL APP_TIMEZONE=UTC
SALON_ID=1 SALON_TIMEZONE=Asia/Colombo
DB_* REDIS_* QUEUE_CONNECTION=redis CACHE_STORE=redis SESSION_DRIVER=redis
MAIL_* (SMTP)
SMS_DRIVER=log|notifylk  SMS_SENDER_ID  SMS_API_KEY  SMS_USER_ID
SMS_TEST_NUMBERS=   # staging: only these numbers receive real SMS
SENTRY_LARAVEL_DSN= (optional)
```

5. Backups & monitoring

- MySQL dump nightly at 02:00 local, compressed, 14 daily + 8 weekly copies, copied off-server (Azure Blob)
- Monthly restore test into staging
- Uptime check on `/up` every 5 min (UptimeRobot or similar), alert to Webxkey WhatsApp/email
- Error tracking: Sentry (or Laravel log + daily email digest)
- Laravel Pulse/Horizon optional for queue visibility

6. Domain & SSL

- Booking URL on client's domain, e.g. `book.kvsaloon.lk` (CNAME/A to server) — TBC with client
- Let's Encrypt via Certbot, auto-renew

7. Go-live runbook

☐ Production `.env` set, `APP_DEBUG=false`
☐ Real services, prices, staff, schedules entered and checked by owner
☐ Seeder NOT run on production
☐ SMS sender ID approved and tested to owner's phone
☐ Backup job ran once and restore tested
☐ Uptime + error monitoring active
☐ QR code / link shared on salon socials and Google Business profile
☐ Staff trained (30-minute session), short guide sent
☐ Support contact and hours agreed

SECTION 11

Open Questions for KV Saloon

Build can start with the defaults below. Update this file (and the related doc) as answers arrive.

#	Question	Default used until answered	Affects	Answer
Q1	How many staff, and which services does each perform?	4 staff (demo)	Seed, skills	
Q2	Full service list with durations, cleanup time and prices	Demo list	Services	
Q3	Opening hours per day; public holiday policy	Mon–Sat 09–19, Sun 10–16	Opening hours	
Q4	Can one booking have services by **different** staff?	No — same staff, back to back (BR-4.1)	Business rules	
Q5	Which assignment rule do you prefer?	Least booked today	BR-5	
Q6	Require SMS code (OTP) to confirm online bookings?	Yes	BR-6	
Q7	Latest time customers can cancel/reschedule online	3 hours before	BR-7.3	
Q8	How far ahead can customers book? Minimum notice?	30 days / 1 hour	BR-1	
Q9	Reminders by SMS only, or WhatsApp too? Times?	SMS, 24h + 2h	BR-8	
Q10	Which SMS provider/sender ID? (we can register one)	Notify.lk, sender "KVSaloon"	Notifications	
Q11	Should customers see staff photos and names?	Yes	Portal	
Q12	Any limits beyond staff (chairs, rooms, equipment)?	None	Availability	
Q13	Booking domain and logo/colours	Subdomain of Webxkey for staging	Branding	
Q14	Gender-specific services or staff?	None	Eligibility	
Q15	Can staff mark their own appointments completed?	Yes	Staff panel	
Q16	Existing customer list to import (Excel)?	No import	Phase 2	

Decisions that change rules must also be recorded as an ADR in `docs/decisions/`.

SECTION 12

AI Agent Playbook (how Webxkey builds with agents)

This project is built with several AI coding agents (Claude Code, OpenAI Codex, Cursor, GitHub Copilot, Gemini CLI, etc.). Agents are fast but forget context between sessions and will guess when unsure. This playbook keeps every agent on the same path.

1. Principles

1. **Docs are the memory.** Agents read `AGENTS.md` + the docs a task names. Anything decided in chat must be written into the docs, or it is lost.
2. **One task, one branch, one PR.** Never ask an agent to "build the booking system". Give it one `BK-xxx` task.
3. **Tests before trust.** For anything in `app/Domain`, the agent writes tests from the business rules first, then code.
4. **Humans merge.** Agents open PRs; Farhan or the partner reviews and merges.
5. **Different agent reviews.** Code written by one agent is reviewed by another (or a fresh session) using the review checklist.

2. Agent file setup (in repo root)

File	For	Content
`AGENTS.md`	Codex, Cursor, Copilot, Gemini, Jules, most agents	Main rules (already written)
`CLAUDE.md`	Claude Code	One line: `See AGENTS.md — follow it exactly.` plus any Claude-only notes
`.cursor/rules/main.mdc` (optional)	Cursor	`alwaysApply: true`, points to AGENTS.md
`.github/copilot-instructions.md` (optional)	Copilot	Points to AGENTS.md

Keep **one** source of truth (AGENTS.md); other files only point to it.

3. Roles you can give agents

Role	When	Prompt focus
Planner	Before a complex task (BK-202, BK-301, BK-501)	"Read the docs, propose the class list, method signatures and test list. Do not write code."
Implementer	Each task	Task block + "tests first"
Reviewer	Every PR	Diff + review checklist from 08-coding-standards §6 + relevant BR rules
Tester	After M3, M4	"Write Playwright tests for UAT checklist items …"
Doc keeper	End of milestone	

Role	When	Prompt focus
		"Compare code with docs 02/04/05; list mismatches; update docs where code is right"

4. Implementer prompt template

```
You are working on the KV Saloon booking system.
Read AGENTS.md first, then: <docs from the task's "Read first">.

Task: <paste the full BK-xxx block from docs/07-roadmap-and-tasks.md>

Rules:
- Work only on this task. Branch: feature/BK-xxx-<slug>.
- Write tests first for domain logic, named after the BR rule IDs.
- If the docs are unclear or conflict, stop and list your questions. Do not guess.
- Run: php artisan test, ./vendor/bin/pint --test, ./vendor/bin/phpstan, npm run typecheck, npm
run lint.
- Finish with: summary of changes, files touched, tests added, any doc updates, open questions.
```

5. Reviewer prompt template

```
Review this PR for the KV Saloon booking system as a senior Laravel engineer.
Read AGENTS.md and docs/02-business-rules.md (rules: <BR ids>).
Check: correctness vs rules, race conditions, timezone handling, salon scoping,
N+1 queries, missing tests, security (authz, validation, rate limits).
Output: blocking issues, non-blocking suggestions, missing tests. Do not rewrite the code.
```

6. Daily loop

1. Pick next ☐ task with all dependencies ☑ → mark ◐ in `07-roadmap-and-tasks.md`.
2. (Complex tasks) planner session → approve plan.
3. Implementer session → PR.
4. CI green → reviewer agent → human review → squash merge → mark ☑.
5. Merged to `develop` → auto-deploy to staging → quick manual check.
6. Weekly: demo to client on staging; doc keeper pass; update open questions.

7. Working in parallel

Safe to run in parallel (different files): BK-103/BK-601, BK-201/BK-204, BK-401/BK-501 after their dependencies. Never run two agents on tasks touching `BookingService` or migrations at the same time. Use separate git worktrees/branches per agent.

8. Red flags — stop the agent if it

- Edits a merged migration or `.env`
- Changes business rules to make a test pass
- Adds a package not discussed
- Puts booking logic in a controller or React component

- Skips the concurrency test for BookingService
- Uses `now()` inside domain services or stores local time in the DB

APPENDIX A

AGENTS.md — KV Saloon Booking System

Instructions for **any** AI coding agent (Claude Code, Codex, Cursor, Copilot, Gemini, etc.) and human developers working in this repository. Read this file fully before starting any task.

1. What we are building

Phase 1 of the KV Saloon system: **online booking with automatic staff assignment**, an admin/front-desk dashboard, and a staff panel. Later phases (customers, payroll, inventory, reports) will be added to the same codebase, so code must stay modular.

Source of truth, in this order:

1. `docs/02-business-rules.md` — how booking must behave (highest priority)
2. `docs/01-prd-phase1.md` — features, user stories, acceptance criteria
3. `docs/04-database-schema.md` — tables and constraints
4. `docs/05-api-spec.md` — endpoints and payloads
5. `docs/03-architecture.md` — stack, folders, patterns
6. `docs/08-coding-standards.md` — conventions and Definition of Done

If documents conflict, **stop and ask** — do not guess. Record the answer in `docs/decisions/`.

2. Tech stack (do not change without an ADR)

- PHP 8.3, **Laravel 12**
- **Inertia.js + React + TypeScript**, Tailwind CSS
- **MySQL 8**, Redis (queues, cache)
- Tests: **Pest** (backend), Vitest (frontend logic)
- Hosting: Webxkey Azure server (Ubuntu, Nginx, PHP-FPM, Supervisor)

3. How to work on a task

1. Pick **one** task from `docs/07-roadmap-and-tasks.md` (e.g. `BK-301`). Only work on that task.
2. Check its `Depends on` tasks are done.
3. Create a branch: `feature/BK-301-short-name`.
4. Read the docs listed in the task's `Read first` field.
5. Write or update tests **first** for business logic (availability, assignment, overlap).
6. Implement the smallest change that meets the acceptance criteria.
7. Run: `php artisan test`, `npm run typecheck`, `npm run lint`, `./vendor/bin/pint --test`. All must pass.
8. Update docs if behaviour or schema changed, and add a line to `CHANGELOG.md`.
9. Open a PR using the template in `docs/08-coding-standards.md`. A human reviews and merges.

4. Hard rules

- **Never** edit an existing migration that has been merged. Create a new migration.
- **Never** commit secrets. Use `.env` only; update `.env.example` with new keys (empty values).

- All times are stored in **UTC**; display and business rules use the salon timezone (`Asia/Colombo`). Use `CarbonImmutable`.
- Every business table has `salon_id` (multi-tenant ready). Always scope queries by salon.
- Booking writes must go through `BookingService` inside a DB transaction with row locks (see business rules §6). No other code may insert appointments.
- Business logic lives in `app/Domain/*` services/actions, **not** in controllers or React components.
- Money is stored as integer **cents** (`price_cents`), never floats.
- Do not add new packages without noting why in the PR.
- Do not implement features from later phases (POS, payroll, inventory) unless the task says so.

5. Useful commands

```
composer install && npm install
cp .env.example .env && php artisan key:generate
php artisan migrate --seed          # demo salon, 4 staff, services
php artisan serve & npm run dev
php artisan test
php artisan queue:work              # notifications
php artisan schedule:work           # reminders
```

6. Glossary

Term	Meaning
Salon	Tenant (KV Saloon). Phase 1 has one salon, one branch.
Staff	A person who performs services (stylist, beautician). May also be a system user.
Service	Bookable item with duration, buffer, price (e.g. Haircut, 30 min).
Skill	Link between staff and a service they can perform.
Appointment	One customer visit; has one or more items.
Appointment item	One service, by one staff, at a start/end time.
Slot	A possible start time offered to the customer.
Buffer	Cleanup minutes after a service where the staff is blocked.
"Any staff"	Customer lets the system assign a staff member.

APPENDIX B

ADR-001: Laravel modular monolith with Inertia + React

Date: 2026-10-29 · Status: accepted

Context

Small team using AI agents, one salon, Phase 1 in ~7 weeks, more modules later (POS, payroll, inventory, reports). Webxkey hosts on its own Azure server.

Decision

One Laravel 12 application with domain modules under `app/Domain`, Inertia + React + TypeScript front end, MySQL 8, Redis. No separate SPA or microservices. A small public JSON API is kept for the booking flow and a future mobile app.

Consequences

- Fast delivery, one deploy, simple hosting.
- Domain boundaries must be enforced by convention and review (AGENTS.md).
- A native mobile app later can reuse `/api/v1/public`.

APPENDIX B

ADR-002: Prevent double booking with ordered row locks

Date: 2026-10-29 · Status: accepted

Context

MySQL has no exclusion constraints for time ranges, and bookings can include buffers and multiple services, so a unique index on (staff, start) is not enough.

Decision

`BookingService` opens a transaction, locks the candidate `staff` rows with `SELECT … FOR UPDATE` ordered by id, re-checks availability inside the transaction, then inserts. All appointment writes (online, front desk, reschedule, move) go through this service. A concurrency test (BR-6.1) is mandatory.

Consequences

- Correct under concurrent requests; lock scope is per staff, so different staff never block each other.
- "Any staff" bookings lock all eligible staff briefly; acceptable at salon scale.
- Any new write path that bypasses the service is a defect.